

STM32 Cortex-M4 DC Motor Control

상위설계서 / 결과보고서

작성일 2026-07-26

목차

1. 프로젝트 개요
2. 시스템 요구사항
3. 하드웨어 구성
4. 시스템 아키텍처
5. FSM 설계
6. 모듈 설계
7. PWM 제어 설계

1. 프로젝트 개요

프로젝트 목적

STM32 보드와 L293 모터 드라이브를 이용하여 DC Motor 를 제어하는 임베디드 시스템을 구현한다. 버튼 입력을 이용하여 모터를 정방향,역방향,정지 상태를 제어하고, UART 통신과 PWM 을 이용하여 모터의 속도 조절 기능까지 구현하는 것을 목표로 한다.

개발 환경

항목	설명
MCU	STM32F411xE
IDE	Visual Studio Code
Compiler	arm-none-eabi-gcc
Language	C
Debugger	ST-Link
PC 개발환경	Windows 11

2. 시스템 요구사항

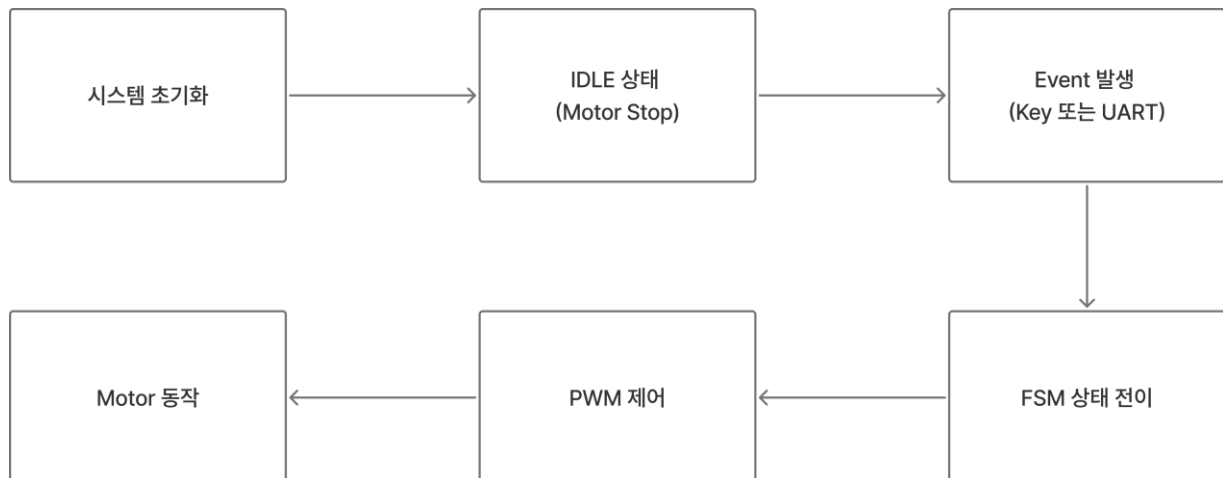
기능 요구사항

- 버튼을 짧게 누를 시 모터를 구동한다.
- 모터가 구동 중일 때 버튼을 짧게 누르면 모터의 회전 방향을 변경한다.
- 버튼을 길게 누르면 모터를 정지한다.
- UART 로 S(정지), F(정회전), R(역회전)을 입력 받아서 모터를 제어한다.
- 속도 정보는 UART 로 1~9 를 입력 받아서 9 단계로 속도를 조절한다

비기능 요구사항

- 버튼 채터링으로 인한 오동작이 발생하지 않아야 한다.
- Long Press 는 3 초 이상 입력 시 발생하도록 구현한다.
- 방향 전환시 탈조 발생을 방지하기 위해, 정지 구간을 둔다.
- PWM 주파수 및 PWM Duty Rate 설정은 실험에 의하여 적절한 값을 적용하도록 한다.
- 단, PWM 의 Duty Rate 이 너무 적으면 모터가 구동되지 않으므로 Durt 범위는 50%~100%로 한다.

동작 시나리오



3. 하드웨어 구성

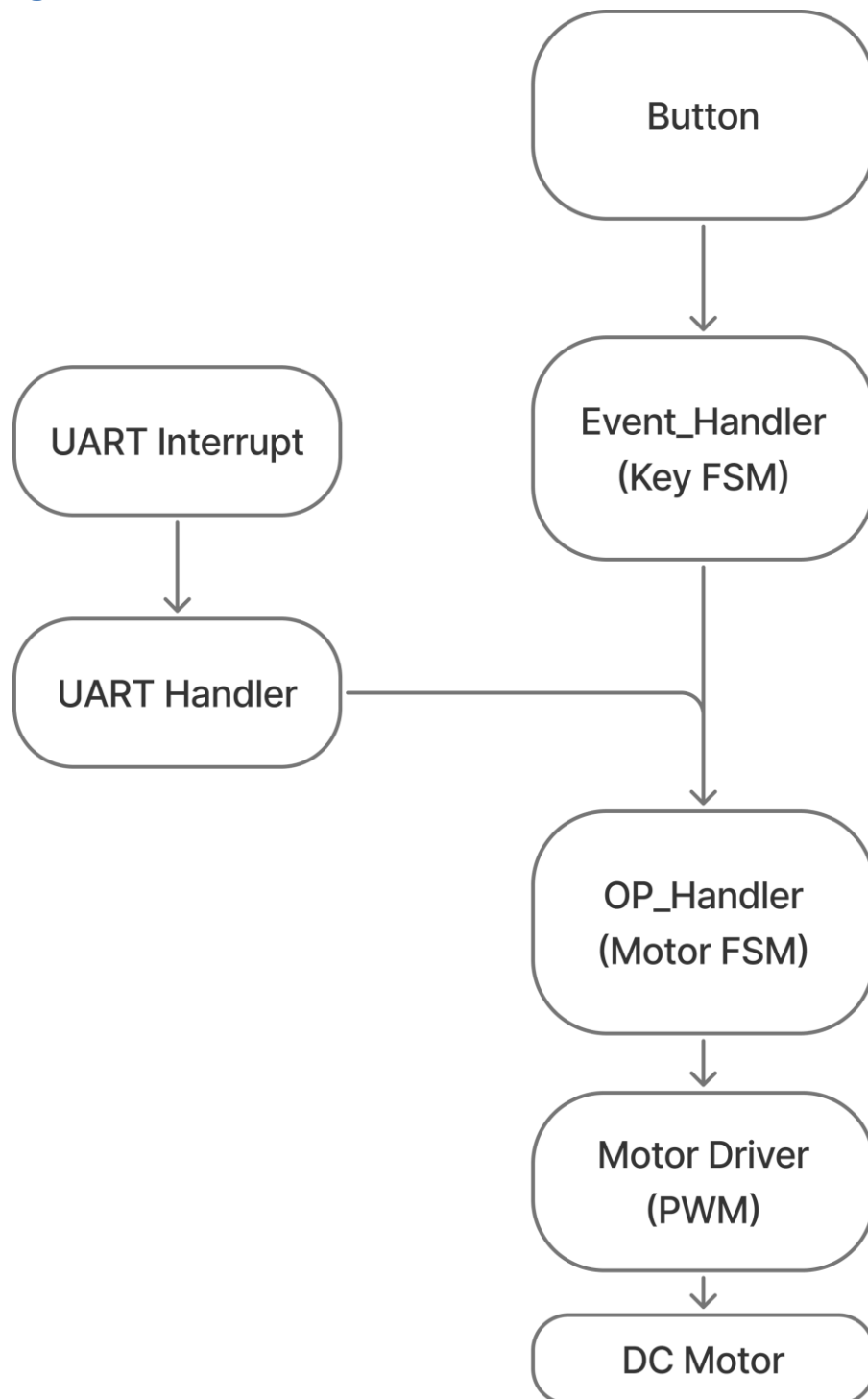
항목	모델	용도
MCU	STM32F411xE	시스템 제어
Motor Driver	L293D	DC Motor 구동
DC Motor	DC Geared Motor	모터 동작
Push Button	On-board User Button	버튼 입력
UART	USART2	PC 와 통신

핀맵 표

Pin	기능
PA0	GPIOA,TIM5_CH1 PWM
PA1	GPIOA,TIM5_CH2 PWM
PA2	USART2_TX
PA3	USART2_RX
PC13	User Button

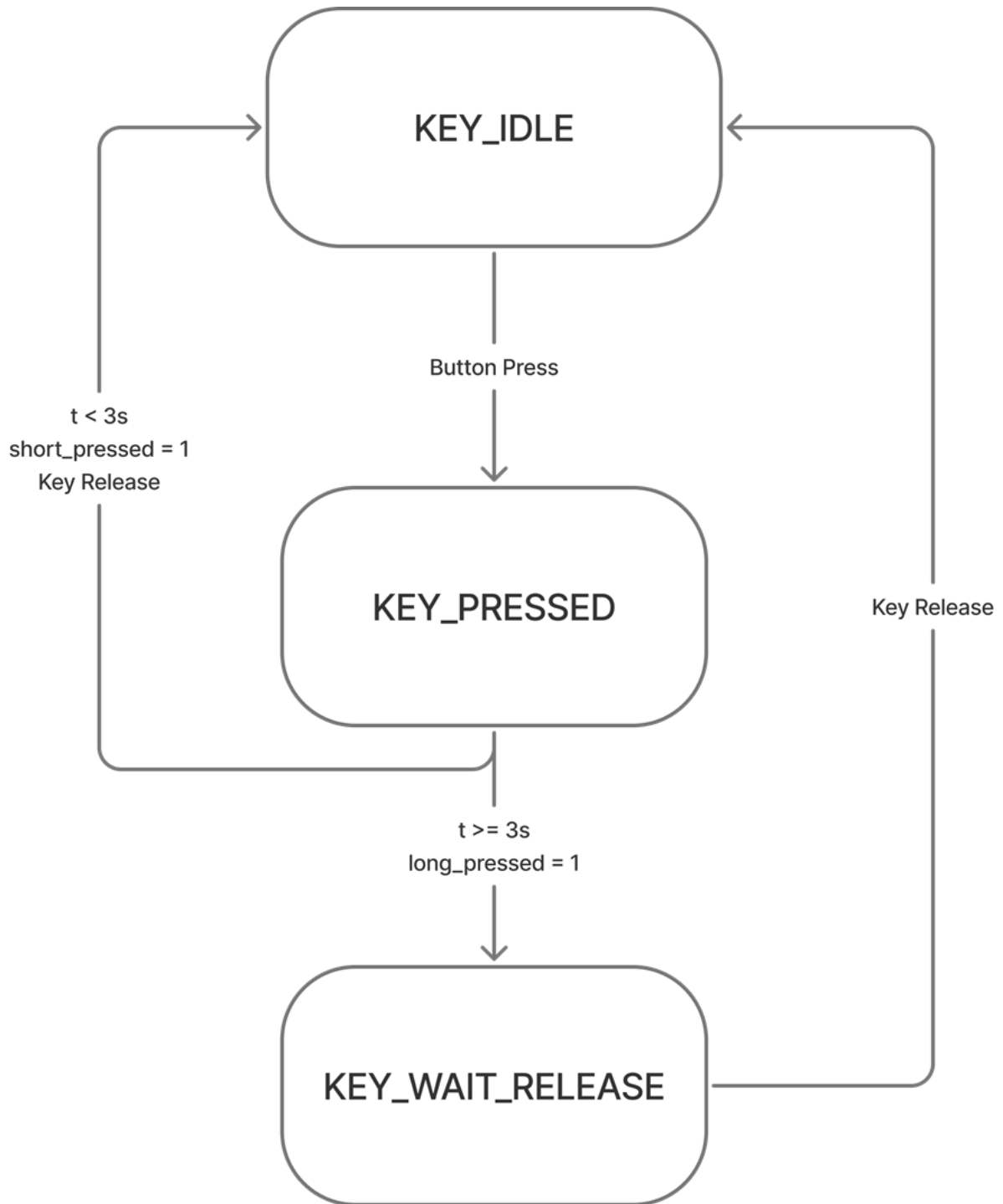
4. 시스템 아키텍처

Block Diagram



5. FSM 설계

5.1 Key FSM



목적

버튼 입력을 Short Press 와 Long Press 이벤트로 구분하여 Motor FSM 에 전달한다.

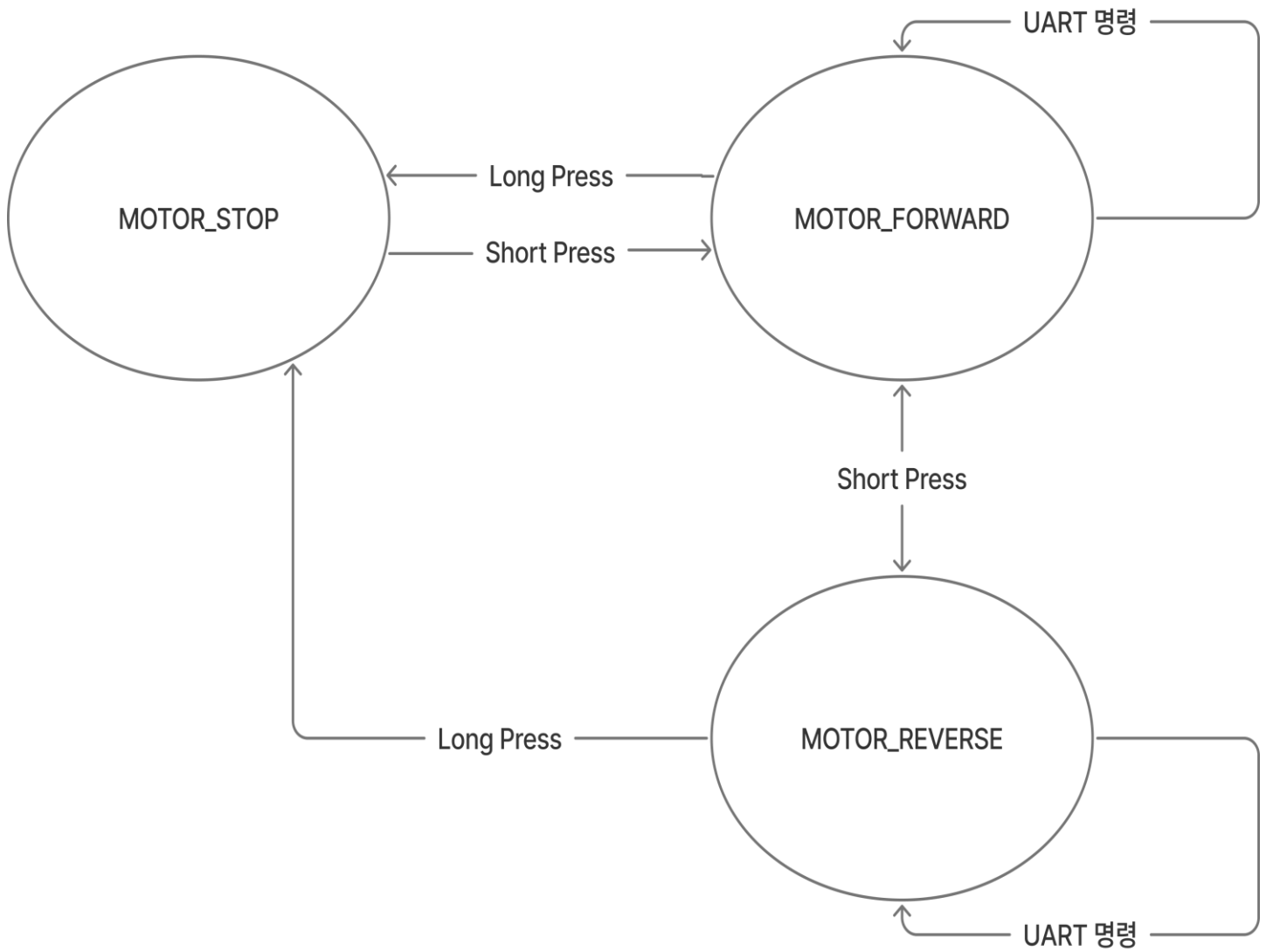
상태 설명

상태	설명
KEY_IDLE	버튼 입력을 대기하는 상태
KEY_PRESSED	버튼이 눌린 상태로, 입력 시간을 측정
KEY_WAIT_RELEASE	Long Press 가 확정된 후 버튼이 해제될 때까지 대기

동작

- ✓ 버튼이 눌리면 KEY_PRESSED 상태로 전이한다.
- ✓ 3 초 미만 해제 시 short_pressed 플래그를 발생시킨다.
- ✓ 3 초 이상 유지 시 long_pressed 플래그를 발생시키고 KEY_WAIT_RELEASE
상태로 전이한다.
- ✓ 버튼이 해제되면 KEY_IDLE 상태로 복귀한다.

5.2 Motor FSM



목적

Key FSM 또는 UART Event로부터 전달받은 이벤트를 해석하여 모터의 동작 상태를 제어한다

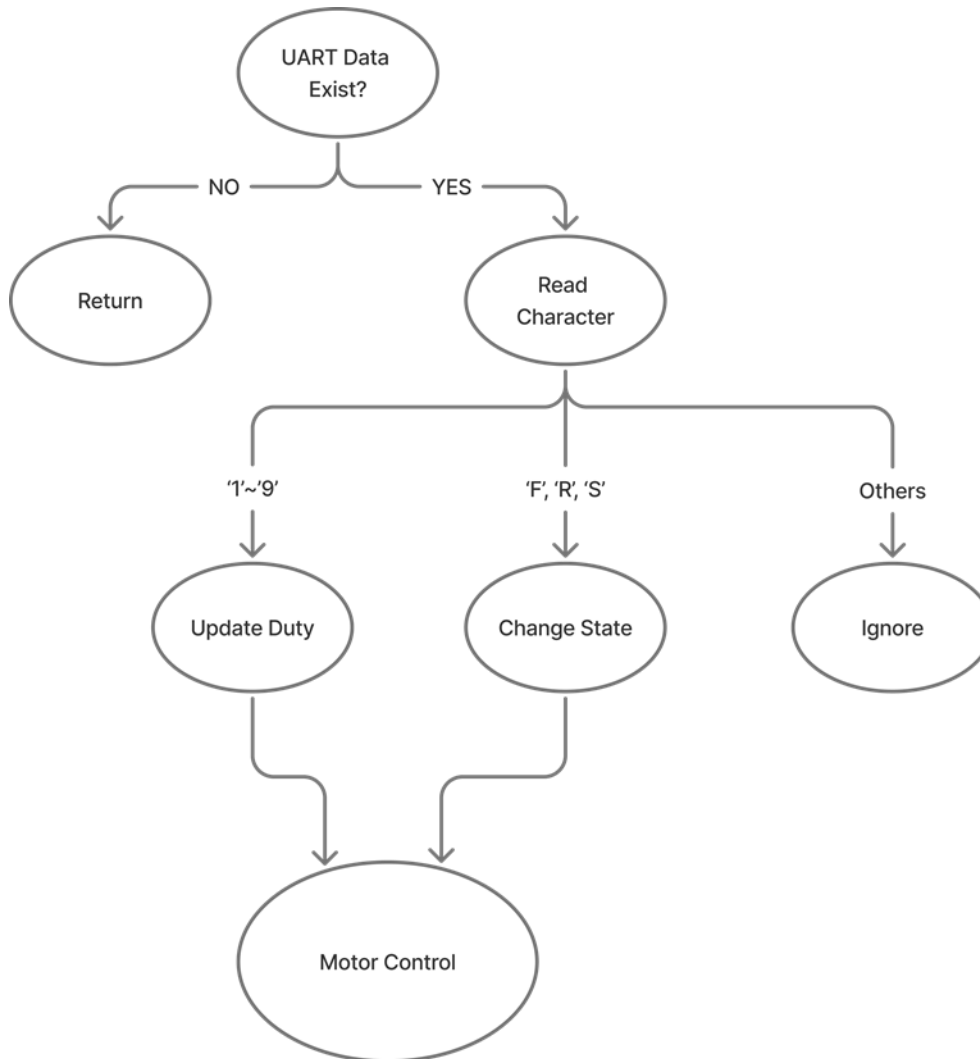
상태 설명

상태	설명
MOTOR_STOP	모터 정지 상태
MOTOR_FORWARD	정회전 상태 (CW)
MOTOR_REVERSE	역회전 상태 (CCW)

동작

- ✓ Short Press
 - STOP -> FORWARD
 - FORWARD -> REVERSE
 - REVERSE -> FORWARD
- ✓ Long Press
 - 모든 상태에서 STOP
- ✓ UART 명령
 - 'F', 'f' : 정회전
 - 'R', 'r' : 역회전
 - 'S', 's' : 정지
 - '1'~'9' : 현재 방향을 유지하면서 PWM Duty 만 변경 (60%~100%)

5.3 UART Event Flow



1. UART 데이터 수신 확인
2. 수신 데이터 읽기
3. '1'~'9' -> PWM Duty 변경
4. 'F' -> 정회전
5. 'R' -> 역회전
6. 'S' -> 정지
7. 그 외 입력 -> 무시

6. 모듈 설계

main.c

✓ 역할

- 시스템 초기화
- UART_Handler, Event_Handler, OP-Handler 수행

✓ 주요 함수

함수	설명
Main()	시스템 초기화 및 메인루트 실행
Event_Handler()	KEY FSM 실행
OP_Handler()	MOTOR FSM 실행
Uart_Handler()	UART 명령 처리

key.c

✓ 역할

- 버튼 입력 감지 및 Debouncing
- Key Event 생성 (short_pressed, long_pressed)

✓ 주요 함수

함수	설명
Key_Poll_init()	GPIO 초기화
Key_Is_Pressed ()	버튼 상태 확인
Key_Get_Pressed()	Debouncing 후 입력 반환

motor.c

✓ 역할

- DC Motor 제어
 - 모터 정지, 정방향, 역방향 회전 제어
- 입력 Duty 에 맞게 PWM 제어

✓ 주요 함수

함수	설명
Motor_Init()	PWM 초기화
Motor_Stop()	모터 정지
Motor_Forward(duty)	정회전(듀티비율)
Motor_Reveerse(duty)	역회전(듀티비율)

timer.c

✓ 역할

- Delay
- Repeat Timer
 - Time Out Check 를 통해 Long press 조건 감지

uart.c

✓ 역할

- DC UART 초기화
- UART 인터럽트 Enable
- UART 데이터 반환
- 디버깅 용 데이터 송신

7. PWM 제어 설계

PWM 원리

PWM(Pulse Width Modulation)은 일정한 주기를 유지하면서 High 구간의 비율 (Duty Rate)을 조절하여 평균 전압을 변화시키는 제어 방식이다. 본 프로젝트에서는 PA0, PA1 에 연결된 TIM5 의 CH1,CH2 를 PWM 모드로 이용하여 DC Motor 의 속도를 제어하였다.

PWM 주파수 설정

- 1 Tick : 1 us
- TIM5_FREQ : 1 MHz
- PWM_FREQ : 10kHz (일반 적인 DC Motor 주파수 : 1kHz ~ 30kHz)

$$f_{PWM} = \frac{f_{CLK}}{(PSC + 1)(ARR + 1)}$$

PSC / ARR 계산

PSC: `TIM5->PSC = (unsigned int)((double)TIMXCLK / TIM5_FREQ + 0.5) - 1;`

ARR: `TIM5->ARR = (unsigned int)((double)TIM5_FREQ / PWM_FREQ + 0.5) - 1;`

$$PSC = \frac{96MHz}{1MHz} - 1 = 95 \quad ARR = \frac{1MHz}{10kHz} - 1 = 99$$

CCR(Duty) 계산

CCR: `(int)((TIM5->ARR + 1) * (duty / 100.));`

$$Duty(\%) = \frac{CCR}{ARR + 1} \times 100$$